

Evil Twin Attacks

A structured study guide covering Evil Twin fundamentals, attack-chain concepts, testing methodology, prevention, and framework mapping.

[!WARNING]

Authorized-lab use only. The techniques described in this guide should be practiced only in an isolated lab or under explicit, documented authorization. Do not target networks, devices, or users you do not own or have permission to test.

This guide picks up where the wireless tooling series left off, but shifts to a genuinely different category of attack. Aircrack-ng, Reaver, and Wifite all worked to recover an existing network's key or PIN — an Evil Twin attack doesn't attack the target network's cryptography at all. It exploits a structural gap in Wi-Fi's trust model: **a client device has no reliable way to cryptographically verify that an access point broadcasting a familiar network name is the legitimate one.** Worth locking in the distinction immediately: **Part 1 covers why this gap exists and how a convincing Evil Twin is actually built — Part 2 covers the practical attack chain, from setup through credential harvesting**, since a working Evil Twin is really several distinct components (a rogue radio, a captive portal, and often a deauthentication attack) working together.

1. Why Client Devices Can't Tell the Difference — The Foundation

A Wi-Fi network is identified to users almost entirely by its **SSID** (network name) — a value any device can broadcast freely. There is no built-in cryptographic binding between an SSID and a specific, verified radio the way, for instance, a TLS certificate cryptographically ties a domain name to a verified key pair.

```
Legitimate AP:  SSID "CoffeeShop-WiFi"  BSSID AA:BB:CC:11:22:33
Evil Twin AP:   SSID "CoffeeShop-WiFi"  BSSID DD:EE:FF:44:55:66
- broadcasts the IDENTICAL name from entirely
  different hardware, often at higher signal
  strength to win the client's preference
```

The single fact this entire guide is built around — most client devices, by default, will list and often automatically reconnect to any network whose SSID matches one previously joined, with no inherent way to distinguish a genuine access point from an impostor broadcasting an identical name. This is precisely why an Evil Twin attack requires no cryptographic cracking whatsoever — it wins by being *trusted*, not by being *decrypted*.

Part 1 — Building a Convincing Evil Twin

2. Reconnaissance — Identifying the Target Network

```
airodump-ng wlan0mon
```

Same reconnaissance step covered in the Aircrack-ng guide's Section 3 - the target's SSID, BSSID, channel, and encryption type (open vs. password-protected) all inform how the rogue AP should be configured to convincingly mimic it

3. Standing Up the Rogue Access Point

```
# using hostapd to broadcast a matching SSID:
hostapd hostapd.conf
```

```
# hostapd.conf (minimal example):
interface=wlan1
driver=nl80211
ssid=CoffeeShop-WiFi
channel=6
```

Tools purpose-built for this specific workflow (e.g., airgeddon, wifiphisher) wrap hostapd, DNS/DHCP services, and a captive portal into a single orchestrated setup, directly analogous to how Wifite orchestrates Aircrack-ng/Reaver in the previous guide - the underlying components are the same either way

4. Matching the Legitimate Network's Security Posture

If the LEGITIMATE network is OPEN (no password):

- the Evil Twin is trivially convincing - simply broadcasting the matching SSID as an open network is often sufficient, since client devices frequently auto-connect to previously-trusted open networks with no prompt at all

If the LEGITIMATE network is WPA2/WPA3-PERSONAL (passphrase-based):

- the Evil Twin CANNOT actually replicate the real passphrase without already knowing it - some variants instead broadcast an OPEN version of the same SSID, relying on a captive portal (Section 6) to request the "network password" directly from the victim as part of a fake login/verification step, or specifically target devices that have been forced to forget their saved passphrase via other means

5. Cloning the BSSID (MAC Address) for Added Convincingness

```
macchanger -m DD:EE:FF:44:55:66 wlan1
```

Spoofing the legitimate AP's actual MAC address makes the rogue AP appear identical even to tools/users who check BSSID rather than SSID alone — though this alone doesn't resolve the fundamental ambiguity of two devices transmitting under the same identity simultaneously, and can itself become a detectable anomaly (see Section 12).

Part 2 — The Attack Chain

6. Forcing Victim Disconnection — Deauthentication

Rather than waiting for a victim to manually select the rogue network, an attacker commonly forces a disconnection from the legitimate AP first — using the exact same technique covered in the Aircrack-ng guide's Section 7:

```
aireplay-ng --deauth 0 -a AA:BB:CC:11:22:33 wlan0mon
```

```
--deauth 0 → send CONTINUOUS deauthentication frames (0 = no
              limit), keeping the legitimate AP unavailable to
              nearby clients for as long as the attack runs,
              maximizing the chance a victim's device falls back
              to searching for and joining the impostor network
              instead
```

7. Captive Portal Credential Harvesting

The most common payload once a victim connects to the rogue AP: a fake **captive portal** page — the login/splash screen many legitimate public networks present — convincingly replicated to match the expected venue or network:

```
Victim connects → DNS/DHCP served by the rogue AP redirects ALL
requests to a locally-hosted fake login page → victim enters
credentials (network password, personal information, or payment
details, depending on the portal's design) believing they're
completing a routine sign-in → credentials are captured by the
attacker, and the victim is typically redirected to genuine
internet access afterward to avoid raising suspicion
```

8. Traffic Interception Beyond the Portal

Once a victim's device is actually associated with the rogue AP (whether or not they interacted with a captive portal), **all** of their traffic passes through attacker-controlled infrastructure — enabling passive monitoring of unencrypted traffic, DNS manipulation, and active man-in-the-middle interception of any connection not independently protected by its own encryption (e.g., HTTP sites, or TLS connections where the victim can be tricked into ignoring a certificate warning).

9. Why HTTPS Doesn't Fully Neutralize This

A victim connected to an Evil Twin whose traffic is being intercepted still benefits from TLS/HTTPS protecting the *content* of properly-secured connections — but the attacker still learns which domains are being visited (via DNS and SNI), can attempt to downgrade or strip HTTPS on sites that don't enforce it strictly (HSTS), and can present convincing phishing pages for any site the victim is redirected to, entirely independent of whether the real destination site's own TLS configuration is sound.

10. Testing Methodology

1. Confirm authorization explicitly covers Evil Twin/impersonation testing specifically, including any captive-portal credential capture component - this is a materially more invasive technique than passively cracking a key and warrants explicit scope agreement
2. Identify the target network's exact SSID, security posture, and typical client behavior (Section 2)
3. Stand up the rogue AP matching the target's SSID and, where

- relevant, BSSID (Sections 3-5)
4. Force victim disconnection from the legitimate AP if a passive approach isn't yielding connections (Section 6)
 5. Deploy an appropriately scoped captive portal or monitoring setup per the engagement's actual objective (Section 7-8)
 6. Document exactly which credentials/data were captured and from how many distinct victims, as the core reportable finding

11. Legal & Ethical Note

Setting up an Evil Twin against any network or user population without explicit, documented authorization is illegal in most jurisdictions - this includes both the network impersonation itself and any resulting credential/data capture, which in many jurisdictions constitutes a separate, additional offense beyond unauthorized network access alone. Legitimate use is confined to networks and user populations covered by an explicit penetration-testing or security-awareness engagement, or an isolated lab environment with no real victims involved.

12. Prevention & Detection

- Prefer WPA2/WPA3-ENTERPRISE with proper certificate validation enforced on client devices wherever feasible - Enterprise mode's certificate-based server authentication gives clients an actual cryptographic basis for rejecting an impostor AP, unlike SSID matching alone (directly connects to the PKI and Certificates guides)
- Disable client auto-connect behavior for open/low-trust networks, and train users to manually verify network legitimacy with venue staff rather than connecting based on a familiar-looking name alone
- Deploy Wireless Intrusion Detection/Prevention (WIDS/WIPS) capable of flagging duplicate SSID broadcasts, spoofed/duplicate BSSIDs, and abnormal continuous-deauthentication patterns (Section 6's specific signature)
- Use a VPN for sensitive traffic on any untrusted network - this protects data even if the underlying Wi-Fi association itself has been compromised by an Evil Twin
- Enforce HSTS and modern TLS configuration on any service handling sensitive data, reducing the effectiveness of Section 9's downgrade/interception risk even if a victim does connect to a rogue AP

Quick Reference Cheat Sheet

RECON:

`airodump-ng wlan0mon` → identify target SSID/BSSID

STAND UP THE ROGUE AP:

`hostapd hostapd.conf` → broadcast matching SSID
`macchanger -m <target_BSSID> wlan1` → optional BSSID cloning

FORCE VICTIM DISCONNECTION:

`aireplay-ng --deauth 0 -a <target_BSSID> wlan0mon`

HARVEST:

Captive portal replicating the expected login/splash page
DNS/DHCP redirect ensures ALL victim requests reach the portal

Root lesson: this attack requires ZERO cryptographic cracking - it wins purely by exploiting the absence of any binding between a broadcast SSID and a specific, verified radio. A strong WPA2/WPA3 passphrase provides no defense at all against a well-executed Evil Twin, which is exactly why Enterprise-mode certificate validation (not passphrase strength) is the meaningful technical countermeasure.